

LSTM「基于时间的趋势建模」复现总结

2025 MCM Problem C / Team #2500759 · 论文 3.1.2 节

交付对象：3.1.3 节 XGBoost-Bootstrap

复现交付文档 v1.0

2026-09-25

本文是 *LSTM* 模块的完整总结与交付说明。数据文件位于 交付数据/*data*/, 全部口径、指标、限制均在此列明；下一阶段任务书见 提示 *_XGBoost_Bootstrap* 阶段 *.md*。

1 模块定位与一句话总结

论文 3.1.2 节处在链条的中间：加权奖牌 → PCA 降维 → LSTM 趋势建模 → XGBoost-Bootstrap 区间预测 → 突破国家识别 / 小项归因 / DID 因果。它不输出最终奖牌预测，也不做统计检验，只做一件事：

把每个国家“历届金牌数与奖牌总数”这条带噪声、四年一采样、长度不等的序列，压缩成一个能代表其未来走向的趋势值；同时用一条辅助任务把“东道主/备战效应”识别出来。趋势值随后作为 XGBoost 的输入特征，与 5 个主成分、2028 年主场效应一起进入最终预测模型。



2 模型规格（与论文公式一一对应）

表 1：模型规格

公式	内容与实现要点
(1)	主场优势系数 $\alpha_c = \frac{\text{Gold}_c^{\text{host}} - \text{Gold}_c^{\text{avg}}}{\text{Gold}_c^{\text{avg}}}$ 。因果实现：对“目标届 = t ”的样本，只用 $< t$ 的历史主办届与历史成绩；从未主办则为 0。（原文未说明口径，此处为避免信息泄漏而固定）
(2)	备战期效应 $\text{Prep}_t = 1$ 若 $t \in [t_{\text{host}} - 8, t_{\text{host}}]$ （8 年 = 两个奥运周期）
(3)	输入向量 $\mathbf{x}_t = [\text{Gold}_t, \text{Total}_t, \text{Host}_t, \text{Prep}_t, \alpha_c]$ ，每步 5 维
(4)	双通道 LSTM 各 1 层： $\mathbf{h}_t^{\text{medal}} = \text{LSTM}_{\text{medal}}([\text{Gold}_{t-k}, \text{Total}_{t-k}]_{k=0}^{n_{\text{steps}}})$ ， $\mathbf{h}_t^{\text{host}} = \text{LSTM}_{\text{host}}([\text{Host}_{t-k}, \text{Prep}_{t-k}, \alpha_c])$ ， $\hat{y}_t = \text{FC}(\mathbf{h}_t^{\text{medal}} \oplus \mathbf{h}_t^{\text{host}})$ ； $n_{\text{steps}} = 3$
(5)	分层损失 $\mathcal{L} = \lambda_1 \mathcal{L}_{\text{medal}} + \lambda_2 \mathcal{L}_{\text{host}}$ ， $\lambda_1 = 0.7, \lambda_2 = 0.3$ ； $\mathcal{L}_{\text{medal}}$ 为 Gold/Total 的 MSE（标准化尺度）， $\mathcal{L}_{\text{host}}$ 为以目标届主办状态为标签的 BCE（分类头 sigmoid 输出）
(6)	$\text{WMAE} = \frac{\sum_c w_c y_c - \hat{y}_c }{\sum_c w_c}$ ， $w_c = \text{真实当届金牌数}$ （ $w_c = 0$ 的样本不计入；另出 $w_c + 1$ 平滑对照）

训练协议：Adam(lr = 10^{-3})、batch = 16、dropout = 0.2、weight_decay = 10^{-5} 、最多 200 epoch；早停判据 = 验证集 mean(WMAE_{Gold}, WMAE_{Total})，恢复验证集最优权重；hidden_size = 32、patience = 6（论文点名的配置）；随机种子 0-4，交付值为 5 种子均值，标准差随附。

3 数据口径（下游必须沿用）

表 2：数据口径

项	取值
时间范围	1952–2024，共 19 届（1896–1948 与 1906 未使用）
国家口径	合并历史政治实体：苏联/独联体/ROC→俄罗斯，东德/西德/德国联队→德国，捷克斯洛伐克→捷克，南斯拉夫/南联盟/塞黑→塞尔维亚，福尔摩沙/台湾→中华台北；同一届多实体求和；非国家代表队剔除
国家数量	82 个（1952 年后至少 5 届有奖牌记录）
面板	82×19 完整网格，未上榜届补 0
样本对齐	窗口 = 连续 3 届 → 预测下一届（ $t \rightarrow t+1$ ）——修正原文同届对齐的信息泄漏
划分	训练目标届 ≤ 2016 （1148）；验证目标届 = 2020, 2024（164）；预测 = 2028（82）
标准化	Gold/Total 按训练窗口统计量标准化后算损失；交付的趋势值已反变换回原始奖牌尺度

4 复现结果

4.1 预测精度（验证集 2020/2024, 5 种子均值）

表 3: 交付配置 (32×6, main 变体) 的评估指标

集合	目标	WMAE	MAE	MSE	RMSE	MAPE
验证	Gold	3.562	1.776	11.61	3.41	85.3%
验证	Total	8.578	4.102	65.44	8.09	125.4%
训练	Gold	—	1.756	—	—	—
训练	Total	—	4.342	—	—	—

验证集金牌 **pred-true** 相关系数 $r = 0.897$ (回归斜率 0.913, 略向均值收缩)。若只看真实金牌 ≥ 5 的国家: 金牌 MAPE 32.1%、总数 MAPE 23.8%。**MAPE** 在本场景不可用 (大量国家金牌为 0), 建议下游只用 MAE / WMAE / RMSE。

4.2 2028 年趋势值（前 12 名，对照 2024 实际）

表 4: 2028 趋势值 (5 种子均值) 与 2024 实际

国家	趋势 Gold	趋势 Total	2024 实际 (G/T)
美国	38.7	103.5	40 / 126
中国	33.8	91.0	40 / 91
英国	28.3	77.5	14 / 65
日本	25.2	69.0	20 / 45
法国	20.3	56.5	16 / 64
澳大利亚	20.3	56.3	18 / 53
俄罗斯	18.9	53.0	— (禁赛, 数据标记为 0)
德国	16.6	47.3	12 / 33
意大利	13.9	39.9	12 / 40
荷兰	12.9	37.1	15 / 34
韩国	10.2	30.3	13 / 32
加拿大	8.1	23.9	9 / 27

4.3 调参网格（复现论文图 3 / 图 4）

跑满 $\text{hidden} \in \{32, 64, 128\} \times \text{patience} \in \{6, 7, 8, 9, 10\}$ 共 15 组、每组 5 个种子。结论:

- 配置间差异 (5 种子均值极差): Gold 0.87、Total 1.89; 而同一配置换种子的极差平均 1.06/2.78、最大 2.81/6.97——噪声比 “超参数效应” 大 2–3 倍;
- 配对 t 检验 (同种子相减, $n = 5$, 临界值 2.776): 所有两两比较 $|t| < 2.5$, 全部不显著;

- 论文图 3 那种“每个 patience 都不同”的锯齿，蒙特卡洛模拟显示可由**单次运行噪声**解释（Total 极差 5.28 落在我们单次运行分布 [2.52, 7.35] 内）；
- 若采用“恢复最优权重”协议，hidden = 64/128 时 patience 完全失效（各取值结果完全相同）。

4.4 p 值消融：原文 $\hat{p}_i = 0.1$ 的实证答案

原文把 $\hat{p}_i$ 写成常数 0.1。三种实现对比（5 种子）：

表 5: $\hat{p}$ 的实现方式对预测与主办识别的影响

变体	val WMAE_Gold	val WMAE_Total	host AUC	平均训练轮数
main（标准 BCE）	3.562	8.578	0.904	13.0
A（bias 初始化 $\log \frac{0.1}{0.9}$ ）	3.396	8.193	0.957	18.4
B（标签平滑 $\varepsilon = 0.1$ ）	3.444	8.204	0.944	17.8

结论：把 0.1 理解为分类头 **bias** 先验（防止 1.2% 的稀有正类塌缩）最合理，且该歧义**不影响主要结论**。交付模型使用 main 变体（与论文写法一致）。

5 交付数据清单

表 6: 交付文件（位于 交付数据/data/，附 SHA256 校验和）

文件	行数	用途与关键列
lstm_trend_history.csv	1312	XGBoost 训练/验证特征： Country,Year,Gold_trend(+std),Total_trend(+std),Host_prob
lstm_trend_2028.csv	82	2028 预测行特征： Gold_trend_2028(+std),Total_trend_2028(+std),Host_prob_20
base_panel_country_year.csv	1558	主键表 + 真实值 + 主场特征： Country,Year,Gold,Total,IsHost,Prep,Alpha_c
host_years_by_country.csv	82	各国主办年份（含 2028 美国、2032 澳大利亚）
lstm_run_config.json	—	交付模型配置与全部口径（可直接引用到论文附录）
lstm_metrics.csv	1	交付配置的 5 种子评估指标汇总
manifest.json	—	文件清单、行数、SHA256

合并方式：以 (Country, Year) 为键左连接 base_panel 与 lstm_trend_history；2028 预测行的键只有 Country。历史趋势值只覆盖 **1964–2016** 与 **2020、2024**（前 3 届因缺少 3 届窗口无法产出趋势值）。

6 下游使用要点

1. **趋势值只是特征，不是最终预测。**例：英国 2028 趋势 28.3，而其 2024 实际仅 14（趋势是长期水平的正则化估计）。
2. **XGBoost 的三类输入：**本交付的趋势值；5 个 PCA 主成分（3.1.1 节，尚未复现，需补做：金/银/铜按 50/30/20 加权后对 45 个大项做 PCA）；目标届主场效应（IsHost/Prep/Alpha_c）。
3. **为 3.2 节预留接口：**Bootstrap 阶段必须保存完整的 1000×82 预测样本矩阵（金牌、奖牌总数各一份），因为 3.2 节要对“点估计 ≥ 1 的 Bootstrap 样本”做 Wilcoxon 符号秩检验。
4. **历史趋势值含 in-sample 部分：**1964–2016 落在 LSTM 训练集内，2020/2024 才是真正样本外；若严格要求，可对 LSTM 也做滚动时间序列 CV 重新生成趋势值。
5. **不要使用 Host_prob：**host 分类任务训练集仅 14 个正样本，概率绝对水平不可信（三变体间 0.05–0.19 不等）。

7 质量、限制与已踩的坑

- **种子噪声大：**同一配置换种子，验证 WMAE 可差 2–7 枚奖牌；交付值已取 5 种子均值。
- **俄罗斯 2024 数据伪影：**2024 被禁赛（数据表无 Russia/ROC 行 $\Rightarrow$ 记 0），LSTM 按历史趋势给出 26.7 金 / 73.1 牌，是验证集最大误差源；下游会继承该异常。
- **小国趋势值可能恒定：**窗口内三届均为 0 奖牌 $\Rightarrow$ 输入完全相同 $\Rightarrow$ 输出完全相同（如阿尔及利亚 1964/1968/1972）。
- α_c **重尾：**最大到 7.34（比利时，因其历史均值极低）；交付表保留原值，下游使用前建议标准化或截断。
- **已修复的真实数据坑：**hosts 表 2020 行写作 "Tokyo, Japan (postponed to 2021 …)"（导致日本主办/备战标记丢失）；NOC 名称尾部空格；hosts 的 United Kingdom 与奖牌表的 Great Britain 不一致；奖牌表只收录获奖国家（必须补 0）。
- **与论文不可直接比较：**论文表 5 报告 Gold 测试集 $MSE = 6.83$ 、 $MAE = 0.52$ 、 $MAPE = 14.47\%$ ；其国家口径、对齐方式、划分与 MAPE 口径均未公开，本复现按“无泄漏 + 时间留出”重建，误差更大属预期。

8 复现方式

命令	说明
<code>python -m pip install torch --index-url https://download.pytorch.org/whl/cpu</code>	安装 CPU 版 PyTorch (~200 MB)
<code>python lstm_312.py --variant all --seeds 0 1 2 3 4 --hidden 32 --patience 6</code>	主实验 (交付数据来源)
<code>python lstm_312.py --grid --hiddens 32 64 128 --patiences 6 7 8 9 10 --select last</code>	调参网格 (15 配置 × 5 种子)
<code>python export_delivery.py</code>	重新生成交付数据
<code>python verify_patience_from_single_run.py</code>	可选: 单次训练派生整条 patience 扫描的一致性验证

全部随机种子固定 (random/numpy/torch), CPU 单次训练约 4-5 秒, 完整复现约 5 分钟。